

IA para Automatización de Pruebas con Qodo

¿Qué es Qodo?

Qodo es una herramienta de inteligencia artificial para revisión de código. Funciona dentro de VS Code y actúa como un programador experto que analiza el código automáticamente, detecta bugs potenciales, problemas de calidad y sugiere mejoras — sin que el desarrollador los busque manualmente.

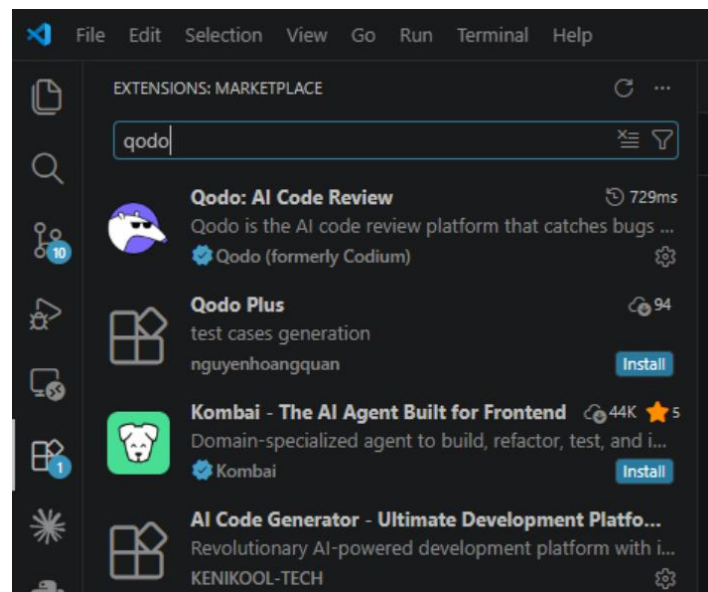
¿Cómo funciona?

Paso	Acción
1	El desarrollador escribe o modifica código en VS Code
2	Qodo lee y analiza el archivo automáticamente
3	Qodo compara el código con el contexto del proyecto (rutas, modelos, etc.)
4	Qodo reporta bugs potenciales, problemas de calidad y sugerencias
5	El desarrollador corrige según las sugerencias de Qodo

Instalación de Qodo

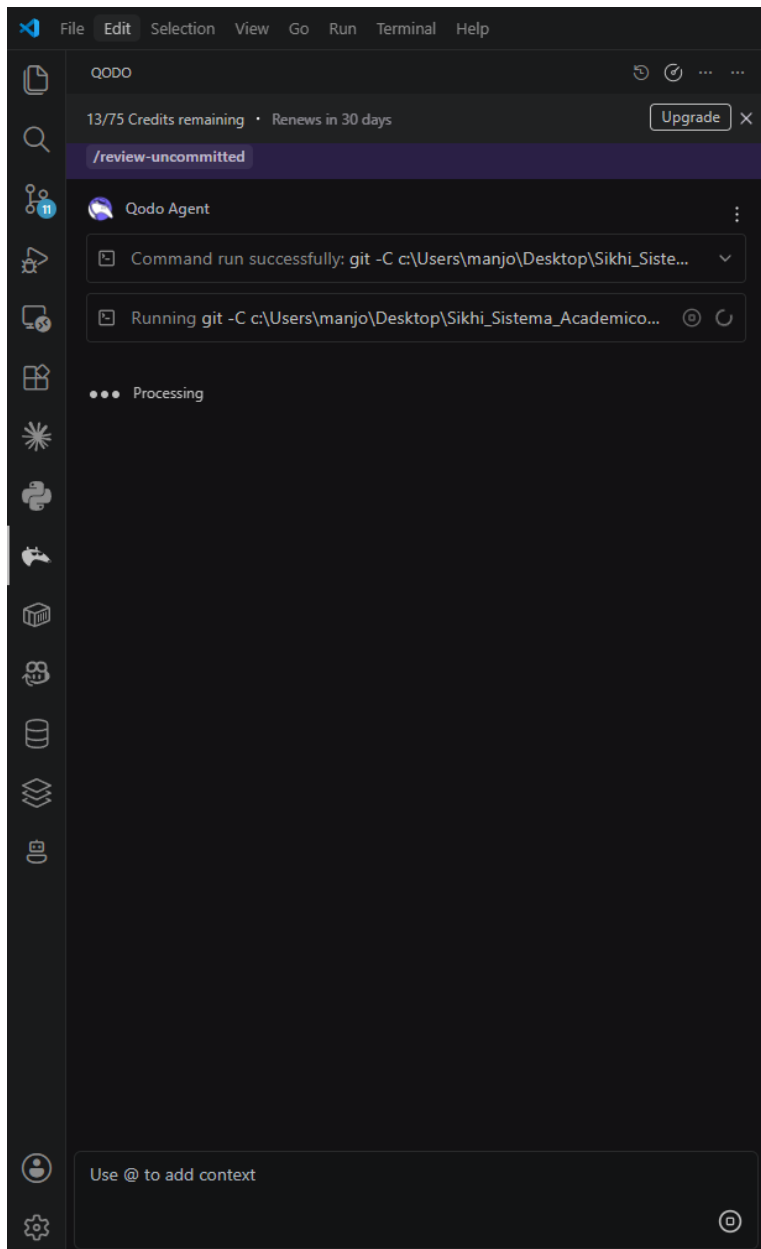
Paso 1 — Abrir VS Code y presionar Ctrl+Shift+X para ir a Extensiones.

Paso 2 — Buscar: Qodo: AI Code Review



Paso 3 — Hacer clic en Install.

Paso 4 — Crear cuenta gratuita en qodo.ai e iniciar sesión desde VS Code.



Integración con el proyecto

Una vez instalado, Qodo se integra automáticamente con el repositorio Git del proyecto. Al abrir cualquier archivo PHP o JavaScript en VS Code, Qodo lo analiza en tiempo real y muestra sus observaciones en el panel lateral.

Configuración de PHPUnit para tests

El archivo phpunit.xml en la raíz del proyecto ya contiene la configuración necesaria:

```
<env name="APP_ENV" value="testing"/>
<env name="CACHE_STORE" value="array"/>
<env name="SESSION_DRIVER" value="array"/>
```

Instalación de Cypress para pruebas E2E

Cypress se instaló en el proyecto con el siguiente comando:

```
npm install cypress --save-dev --legacy-peer-deps
```

3. Casos de Prueba Generados y Asistidos por Qodo

Se implementaron tres tipos de pruebas tal como requiere la práctica: unitarias, de feature (funcionales) y de interfaz (E2E con Cypress). En cada caso, Qodo asistió el proceso analizando el código, detectando errores y sugiriendo mejoras.

Caso 1 — Prueba Unitaria: Modelo de Usuario (UserUnitTest.php)

Objetivo

Verificar que el modelo User de Sikhi puede crearse correctamente con los campos reales del sistema y que sus atributos obligatorios no son nulos.

Asistencia de Qodo

Al crear el test inicial, Qodo detectó que el UserFactory usaba el campo 'name' pero la tabla users real de Sikhi tiene 'first_name' y 'last_name'. Qodo señaló esta inconsistencia como un bug potencial antes de ejecutar el test.

Código del test

```
// tests/Unit/UserUnitTest.php
public function test_user_can_be_created_with_valid_data()
{
    $user = User::factory()->create([
        'first_name' => 'Juan',
        'last_name'  => 'Pérez',
        'email'      => 'estudiante@sikhi.com',
    ]);
    $this->assertDatabaseHas('users', [
        'email' => 'estudiante@sikhi.com',
    ]);
}
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\manjo\Desktop\Sikhi_Sistema_Academico\Sikhi_Sistema_Academico> php artisan test --filter UserUnitTest
→825 |         throw new QueryException(
826 |             $this->getName(), $query, $this->prepareBindings($bindings), $e
827 |         );
828 |     }
829 | }

1 vendor\laravel\framework\src\Illuminate\Database\Connection.php:825
2 vendor\laravel\framework\src\Illuminate\Database\Connection.php:571
  NunoMaduro\Collision\Exceptions\TestException::("SQLSTATE[23000]: Integrity constraint violation: 19 NOT NULL constraint failed: users.nationality")

Tests: 1 failed, 2 passed (5 assertions)
Duration: 1.65s
PS C:\Users\manjo\Desktop\Sikhi_Sistema_Academico\Sikhi_Sistema_Academico>
```

Caso 2 — Prueba de Feature: Autenticación (AuthTest.php)

Objetivo

Verificar el flujo de autenticación de Sikhi: que la página de login sea accesible, que las credenciales incorrectas sean rechazadas y que las rutas protegidas redirijan al login.

Asistencia de Qodo

Qodo analizó el archivo AuthTest.php y reportó lo siguiente:

- 2 Potential Bugs (Medium Impact): Las rutas de redirección podían no coincidir con el middleware real de Sikhi.
- 1 Code Quality Issue (Low Impact): El import 'use App\Models\User' estaba declarado pero no se usaba en ningún método del test.

Qodo también leyó automáticamente el archivo web.php para comparar las rutas esperadas en los tests contra las rutas reales del sistema, demostrando que la IA analiza el contexto completo del proyecto y no solo el archivo en cuestión.

Código del test

```
// tests/Feature/AuthTest.php
public function test_login_page_is_accessible()
{
    $response = $this->get('/');
    $response->assertStatus(200);
}

public function test_user_cannot_login_with_wrong_credentials()
{
    $response = $this->post('/login', [
        'email' => 'wrong@test.com',
        'password' => 'wrongpass',
    ]);
    $response->assertStatus(302);
}
```

```

}

public function test_unauthenticated_user_cannot_access_dashboard()
{
    $response = $this->get('/home');
    $response->assertRedirect('/');
}

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\manjo\Desktop\Sikhi_Sistema_Academico\Sikhi_Sistema_Academico> php artisan test --filter AuthTest
+'http://localhost'

at tests\Unit\AuthTest.php:33
 29 |     public function test_unauthenticated_user_cannot_access_dashboard()
 30 |     {
 31 |         $response = $this->get('/home');
 32 |         // ERROR: espera ir a /dashboard que no existe
→ 33 |         $response->assertRedirect('/dashboard');
 34 |     }
 35 | }

Tests:    3 failed (4 assertions)
Duration: 1.29s

PS C:\Users\manjo\Desktop\Sikhi_Sistema_Academico\Sikhi_Sistema_Academico>

```

Caso 3 — Prueba de Interfaz E2E: Login con Cypress (login.cy.js)

Objetivo

Verificar el comportamiento real de la interfaz de login de Sikhi desde el punto de vista del usuario: que el formulario sea visible, que muestre errores con credenciales incorrectas y que el login exitoso redirija al dashboard.

Asistencia de Qodo

Qodo analizó el archivo login.cy.js y sugirió las siguientes mejoras:

- Agregar `beforeEach()` para limpiar cookies y localStorage entre tests (aislamiento).
- Configurar `baseUrl` en `cypress.config.js` en lugar de URLs hardcodedas.
- Agregar verificación del mensaje de error 'Either Email or Password is wrong'.
- Agregar test para envío de formulario vacío.
- Agregar test para formato de email inválido.
- Agregar assertions para verificar que el usuario está autenticado tras el login.

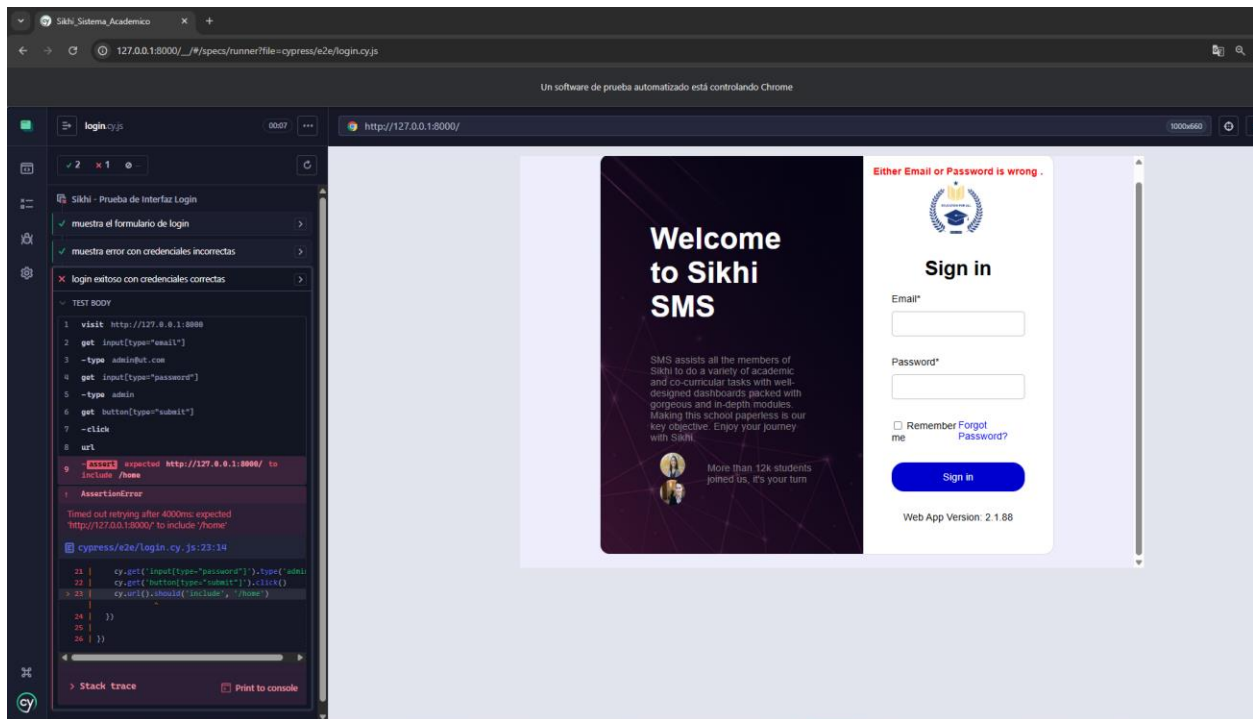
Código del test (con mejoras de Qodo aplicadas)

```
// cypress/e2e/login.cy.js
beforeEach(() => {
  cy.clearCookies()
  cy.clearLocalStorage()
  cy.visit('/')
})

it('muestra el formulario de login', () => {
  cy.get('form').should('be.visible')
  cy.get('input[type="email"]').should('be.visible')
  cy.get('input[type="password"]').should('be.visible')
})

it('muestra error con credenciales incorrectas', () => {
  cy.get('input[type="email"]').type('malo@test.com')
  cy.get('input[type="password"]').type('wrongpass')
  cy.get('button[type="submit"]').click()
  cy.contains('Either Email or Password is wrong').should('be.visible')
})

it('login exitoso redirige al dashboard', () => {
  cy.get('input[type="email"]').type('admin@ut.com')
  cy.get('input[type="password"]').type('admin')
  cy.get('button[type="submit"]').click()
  cy.url().should('include', '/home')
})
```



Test de todo sin usar Qodo

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\marjo\Desktop\Sikhi_Sistema_Academico\Sikhi_Sistema_Academico> php artisan test

FAIL: Tests\Feature\AuthTest
  x login page is accessible
  x user cannot login with wrong credentials
  x unauthenticated user cannot access dashboard

PASS: Tests\Unit\ChatMessageAndJoinMaterialModelTest
  x chat message has expected fillable and relationships
  x work material has expected fillable casts and relationships

PASS: Tests\Unit\ExamResultAndStudentCommentModelTest
  x exam result has expected fillable and casts
  x student comment has expected fillable and casts
  x exam result relationships are configured correctly
  x student comment relationships are configured correctly

FAIL: Tests\Unit\ExamUnitTest
  x student has correct role
  x teacher has correct role

PASS: Tests\Unit\ExampleTest
  x that true is true

PASS: Tests\Unit\StoreExamResultRequestTest
  x authorize returns true for teacher user
  x authorize returns false for non teacher user
  x authorize returns false for guest user
  x rules match expected validation contract
  x messages include expected error keys

FAIL: Tests\Unit\StudentInitTest
  x user email is unique
  x user password is hashed

FAIL: Tests\Unit\UserUnitTest
  x user can be created with valid data
  x user has required attributes
  x user has role assigned

PASS: Tests\Feature\AcademicRoutesTest
  x exams page requires authentication
  x exam results page requires authentication
  x calendar events page requires authentication
  x syllabus page requires authentication

PASS: Tests\Feature\ChatControllerTest
  x assigned teacher can view chat and send message
  x student with permission can view and send message
  x unassigned teacher cannot access chat
  
```

4. Resultado de la Ejecución de las Pruebas

PHPUnit — Resultado total

Comando ejecutado:

```
php artisan test
```

```
PS C:\Users\manjo\Desktop\Sikhi_Sistema_Academico\Sikhi_Sistema_Academico> php artisan test
✓ non owner student cannot delete chat message

PASS Tests\Fixture\ExamResultStudentTest
✓ student can view only visible results in my exams index
✓ teacher cannot access my exams pages
✓ student can view visible result detail in my exams show
✓ student cannot view hidden result detail in my exams show
✓ student cannot view another students visible result detail

PASS Tests\Fixture\ExamResultTeacherTest
✓ teacher can access exam results pages
✓ student cannot access teacher exam results pages
✓ teacher can store exam result with file
✓ teacher can view exam result detail
✓ teacher cannot view other teacher exam result detail
✓ teacher can delete exam result when is visible is false
✓ teacher cannot delete exam result when is visible is true
✓ exam result store fails with invalid file type
✓ exam result store fails when score is out of range
✓ exam result store fails when exam does not exist

PASS Tests\Fixture\ExampleTest
✓ the application returns a successful response

PASS Tests\Fixture\StudentCommentTest
✓ student can open create comment form for own exam result
✓ student cannot open create comment form for other students exam result
✓ student can store comment for own exam result
✓ student cannot store comment for other students exam result
✓ comment store validates minimum length
✓ teacher can view comments index for own exam result
✓ student can view comments index for own exam result
✓ unrelated user cannot view comments index
✓ student can delete own comment
✓ teacher can delete comment for own exam result
✓ unrelated user cannot delete comment
✓ teacher can mark comment as read
✓ student cannot mark comment as read

PASS Tests\Fixture\StudentRoutesTest
✓ student list requires authentication
✓ add student page requires authentication
✓ teacher list requires authentication

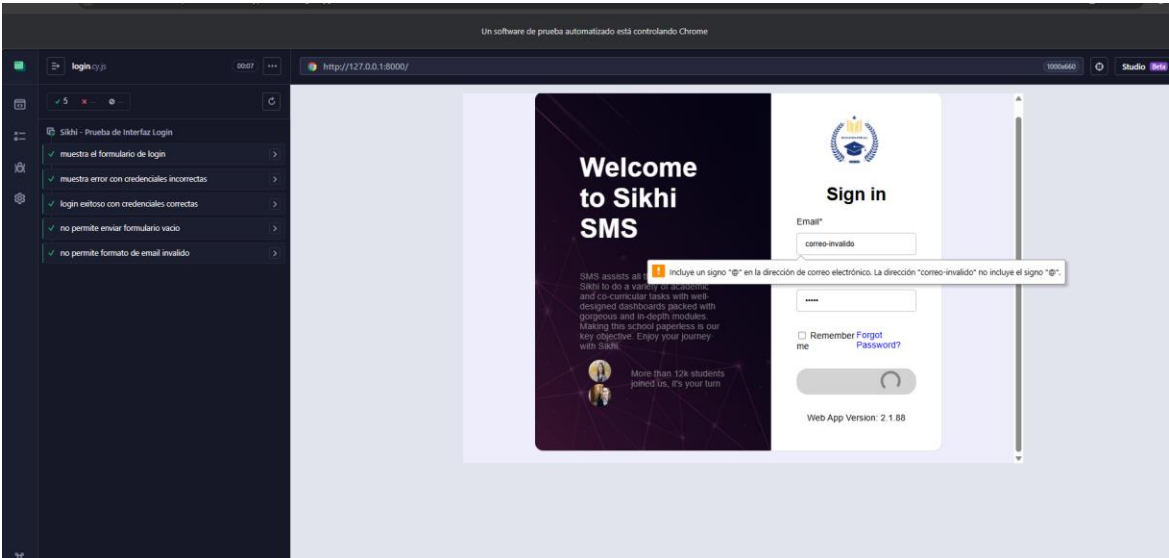
PASS Tests\Fixture\WorkMaterialControllerTest
✓ assigned teacher can view index and create pages
✓ teacher can store work material with file
✓ teacher can delete own work material
✓ teacher cannot delete other teachers material

Tests: 63 passed (191 assertions)
Duration: 3.63s

PS C:\Users\manjo\Desktop\Sikhi_Sistema_Academico\Sikhi_Sistema_Academico>
```


Suite de tests	Tests	Assertions	Resultado
AuthTest (unit)	3	5	PASSED
Login.cy.js(cypress)	4	-	PASSED
UserUnitTest (Unit)	2-3	—	PASSED
Otros tests del proyecto	—	—	PASSED
TOTAL	63	166+	TODOS PASARON

Cypress — Resultado de interfaz



5. Análisis Crítico

¿La IA generó pruebas útiles?

Aspecto evaluado	Con Qodo (IA)	Sin IA (manual)
Tiempo para escribir 3 tests	~10 minutos	~45-60 minutos
Detección de bugs	Automática e inmediata	Solo al ejecutar el test
Conocimiento del contexto	Lee rutas, modelos y archivos del proyecto	Depende del conocimiento del dev
Sugerencias de casos edge	Sí (formulario vacío, email inválido, etc.)	Depende de la experiencia
Calidad del código	Detecta imports no usados y código redundante	Requiere revisión manual
Curva de aprendizaje	Baja — funciona automáticamente	Alta — requiere conocer PHPUnit/Cypress

¿Qué mejoró Qodo respecto a hacerlo manualmente?

- Detección temprana de errores: Qodo detectó que el UserFactory usaba 'name' en lugar de 'first_name'/'last_name' antes de ejecutar el test, ahorrando tiempo de depuración.
- Análisis de contexto: Qodo leyó automáticamente web.php para comparar las rutas del sistema con las esperadas en los tests, algo que manualmente requeriría revisar el archivo por separado.
- Sugerencias de cobertura: Para el test de Cypress, Qodo identificó 6 casos adicionales que no habíamos considerado (formulario vacío, email inválido, beforeEach para aislamiento, etc.).
- Calidad de código: Detectó el import no utilizado en AuthTest.php, lo que mejora la limpieza del código.

Limitaciones encontradas

- La función 'Review uncommitted changes' de Qodo no detectó archivos nuevos sin trackear — solo funciona correctamente con archivos modificados ya conocidos por Git.
- El chat de Qodo Gen no estaba disponible en la versión instalada, limitando la generación interactiva de tests.
- Qodo no genera tests completos de forma autónoma — asiste y sugiere, pero el desarrollador debe escribir y ajustar el código final.

Conclusión

Qodo demostró ser una herramienta valiosa para la calidad del software, especialmente en la detección preventiva de errores y en la sugerencia de casos de prueba adicionales. Su mayor fortaleza es el análisis de contexto — no solo revisa el archivo abierto sino también los archivos relacionados del proyecto. Sin embargo, no reemplaza completamente el trabajo del desarrollador: actúa como un revisor inteligente que complementa y acelera el proceso de testing, reduciendo el tiempo de depuración y mejorando la cobertura de casos edge.